

# P2964R0

## Adding support for user-defined element types in `std::simd`

Published Proposal, 2024-02-09

**This version:**

<http://wg21.link/P2964R0>

**Authors:**

[Daniel Towner](#) (Intel)

[Ruslan Arutyunyan](#) (Intel)

**Audience:**

LEWG

**Project:**

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

---

### Abstract

The paper proposes to support user-defined element types in `std::simd` using customization points

### Table of Contents

- 1 Motivation**
- 2 Understanding customization opportunities and requirements**
  - 2.1 Required customization points
- 3 Creating a customization framework**
  - 3.1 Storage
  - 3.2 Unary and binary operator customization points
  - 3.3 Overloads for free-functions
- 4 Implementation Experience**
  - 4.1 Summary of implementation experience
- 5 Conclusion**

#### References

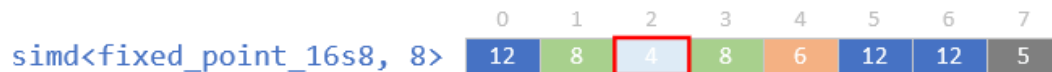
Informative References

## § 1. Motivation

ISO/IEC 19570:2018 (1) introduced data-parallel types to the C++ Extensions for Parallelism TS. [P1928R8] is proposing to make that a part of C++ IS. In the current proposal individual elements must be of a type which satisfies constraints set out by `std::simd` (e.g., arithmetic types, complex-value types). In this document we describe how `std::simd` could allow user-defined element types too, with dispatch of operations to customization functions which handle SIMD implementations for those element.

Being able to have user-defined types in a `std::simd` value is desirable because it allows us to build on top of the standard features of `std::simd` to support SIMD programming in specialised problem domains, such as signal or media processing, which might have custom data types (e.g., saturating or fixed-point types). The idea is that `std::simd` will be used to provide generic SIMD capabilities for performing loads, stores, masking, reductions, gathers, scatters, and much more, and then a set of customization points are provided to allow the fundamental arithmetic operators to be overloaded where needed.

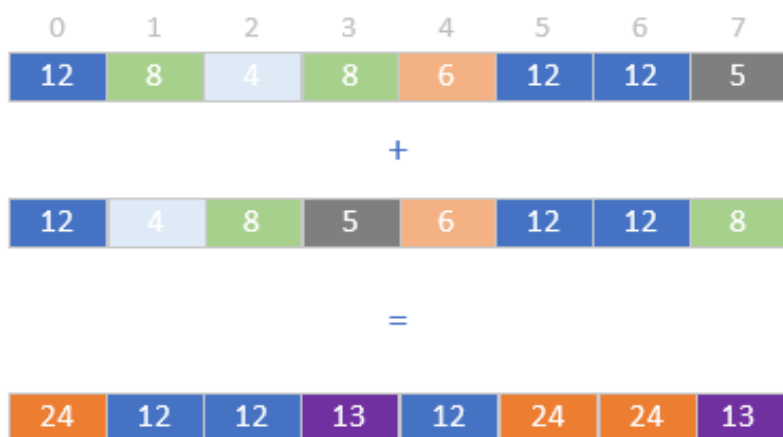
As a concrete example, consider that we might have a fixed-point data type for signal processing called `fixed_point_16s8`. Such a fixed-point data type allows a fractional (non-integer) value to be stored with only a fixed number of digits to represent their fractional part. An instruction set which targets digital signal processing will be likely to have instructions which accelerate the fundamental arithmetic operations for these types and our aim is to allow `simd` values of this underlying fixed-point element type to be created and used. To begin with, a `simd<fixed_point_16s8>` is represented or stored using individual blocks of bits representing each element of the user-defined type within a vector register:



Note that element (highlighted in red rectangle) represents an individual `fixed_point_16s8` element value. The colours and values in each box represent specific bit patterns. Where two elements have the same bit-pattern/colour, they represent the same element value. An operation which moves elements within or across `simd` objects, or to and from memory, will copy the bit patterns. The elements don't change value because they move. In the diagram below a `permute` has been used to extract the even elements of our original example above.



Note that an element representing a specific value in this example (e.g., the dark-blue value 12) continues to represent the value 12 regardless of where it appears within the `simd` object. However, there are also cases where the bit pattern of individual elements does need to be interpreted as a specific value. For example, if we wish to add two `simd<fixed_point_16s8>` values together then we need to define how to efficiently handle the addition of specific bit patterns to form a new bit pattern. In the example below we are adding two `simd<fixed_point_16s8>` values and expecting to get a result which is created by calling some underlying target-specific function:



In this example the `std::simd` objects themselves have no knowledge of this since the elements are of a custom type, so we need to encode that knowledge into a user-defined customisation point. There will be a customisation point in `std::simd` in those places where knowledge of what the bit-pattern in a user-defined `simd` element actually means. Common examples of such points are arithmetic operations or relational comparators. In the example above, a customisation point for `simd::operator+` will be provided which maps onto a suitable hardware instruction to perform a vector fixed-point addition.

Custom types will not always support every possible operator that is exposed by `std::simd`. For example, perhaps a `fixed_point_16s8` doesn't allow division or modulus, in which case `std::simd::operator/` and `std::simd::operator%` must be removed from the overload set entirely.

Putting these mechanisms for defining custom element types together allows us to write generic functions which can then be invoked with both built-in and user-defined types equally easily, using the rich API of `std::simd`, even when the user-defined types are using target-specific customised functions:

```
// Compute the dot-product of a simd value.
template<typename T, typename ABI>
auto dotprod(const basic_simd<T, ABI>& lhs, const basic_simd<T, ABI>& rhs) {
    // The reduction moves are handled bit-wise, while the multiply and addition are delegated
    // to target-specific customisation points. The high-level code is unaltered.
    auto reduce(lhs * rhs, std::plus<>{});
}

...
float dfp = dotprod(simdFloat0, simdFloat1);
fixed_point_16s8 dfxp = dotprod(simdFixed0, simdFixed1);
```

An extended example of a custom element type is given toward the end of this paper, along with our experiences of using it.

This proposed extension of `std::simd` to support custom types applies only to types which can be vectorised as atomic units. That is, a `simd<T>` for a custom type `T` has storage which is the same as an array of those types (i.e., this might be called an array-of-structures). This is in contrast to another way of representing a parallel collection of custom element types in which the structure of the custom element is broken down into individual pieces (i.e., a structure-of-arrays). For example, given `simd<std::pair<A, B>>` the structure-of-array style of customisation would treat it as though it were `std::pair<simd<A>, simd<B>>`. While that could be a valuable feature for future consideration, it is different to what is being proposed in this paper.

In addition to allowing customized element types in `std::simd`, a side-effect of this paper is to set out a way of thinking about the meaning of different `std::simd` operations which makes the division of responsibility of different `std::simd` APIs clear. This makes it easier to discuss the related topic of adding new C++ builtin element types such as `std::byte`, and scoped/unscoped enumerations. For example, as we shall describe later there are specific basis functions which define the fundamental arithmetic behaviour of `std::simd` types, and knowing what those basis functions should be makes it easy to reason about how to support `enum` and `std::byte` types.

Note that a user-defined type, such as our example `fixed_point_16s8`, might provide its own overloaded operators for handling the different types of arithmetic operation that could occur, but it would be too much of a stretch for `std::simd` to be able to define its own operators to mirror those of the underlying type automatically. Suppose we have the following partially-implemented scalar `fixed_point_16s8` type already:

```
struct fixed_point_16s8 {  
  
    fixed_point_16s8 operator+(fixed_point_16s8 lhs, fixed_point_16s8 rhs)  
    { return __intrin_fixed_add(lhs, rhs); }  
    fixed_point_16s8 operator-(fixed_point_16s8 lhs, fixed_point_16s8 rhs)  
    { return __intrin_fixed_sub(lhs, rhs); }  
  
    std::int16_t data;  
};
```

There are three mechanisms in which `std::simd` could add support for `std::simd<fixed_point_16s8>`:

1. Have the compiler reflect on the implementation of the type and automatically extend it into the `std::simd` domain. There are currently no language mechanisms which enable this, so this approach will be discounted.
2. Have `std::simd` generate loops which repeatedly invoke the underlying scalar operators for each `std::simd` element. This relies on the auto-vectorizer being able to turn such code into efficient SIMD code. Since a design goal of `std::simd` is to avoid reliance on auto-vectorization for performance, this approach can also be discounted.
3. Use the idea of customization points (which are widespread in other C++ libraries) to insert special behaviours at those places they are needed. This is the approach used in this proposal.

In the remainder of this proposal we shall look at what parts of `std::simd` need to have customization points to change their behaviour, and which parts are generic enough to leave unchanged.

## § 2. Understanding customization opportunities and requirements

The first step in understanding how `std::simd` can be extended to new element types is to review what operations are provided by `std::simd`, and how those operations must adapt to operating on custom user-defined elements.

Firstly, although [P1928R8] makes no mention of it, there is an implication that the elements in a `std::simd` are trivially copyable. For operations which move elements around - broadcast, permutation, gather, scatter, and so on - it is assumed that when the bits move location within a `std::simd` object they will continue to represent their original value. `std::simd` currently restricts types to be floating-point, integral, or complex-valued (with [P2663R4]), all of which are trivially copyable.

While many operations within `std::simd` will work on trivially copyable custom types there are also places where `std::simd` does need to interpret the meaning of the bits, and it is those that need to be customization points. Each function can be put into one of the following categories:

### Basis

A basis function is one that must be provided as a customization point to allow the underlying element type to be used. An example would be addition; if addition is not provided as a customization point for a user-defined type then

`std::simd` values of that type cannot be added together.

### Custom

A customization function is one that can be implemented generically but which can also be customized to provide a more efficient implementation if one exists. An example of a Custom function would be negate (`operator-`) which could have a default implementation which subtracts from zero, or could be customized if the type provides a faster alternative (e.g., sign-bit flip for a floating-point-like type).

### Copy

A copy function uses the trivially copyable nature of the underlying type and allows bits to be moved from one place to another. The `permute` function is a good example of a Copy function since a `std::simd` of any type can move its elements around within a SIMD value without needing to know what the bits represent.

### Algorithm

An algorithm function uses other functions to implement some feature. If the algorithm relies on a Basis or Custom function which is not provided by the user-defined type then the algorithm function is removed from the overload set.

An algorithm function does not provide a customization point.

The following table lists the key functions in `std::simd`, and what category they fall into. The table allows us to reason about what functions in `std::simd` will just work as they are currently defined, and to separate out those functions which must have customisation points defined in order to allow their behaviour to be changed for custom user-defined types.

| Function                                                                                                    | Type           | Notes                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>simd_mask</code>                                                                                      |                |                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>simd_mask {}</code>                                                                                   | n/a            | Virtually every function in <code>simd_mask</code> will work on user-defined types, with the exception of those listed in the next row. The mask is only dependent on knowing the number of bits in the type, and not on the interpretation of those bits.                                                                                                                                                      |
| <code>simd_mask::operator+</code><br><code>simd_mask::operator-</code><br><code>simd_mask::operator~</code> | Algorithm      | Convert and broadcast a 0, 1 or -1, and perform a <code>simd_select</code> using the mask to choose the appropriate value. Only provided if the convert is available.                                                                                                                                                                                                                                           |
| Constructors                                                                                                |                |                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>basic_simd(U)</code>                                                                                  | Copy           | Broadcast a copy of the bits from represent the scalar source object to every element                                                                                                                                                                                                                                                                                                                           |
| <code>basic_simd(basic_simd&lt;U&gt;)</code>                                                                | Copy/custom    | When U is the same as T, direct copy the bits into place.<br>When U is different, we need a customization point to convert the user-defined elements. If no customization point is available, no conversions will be allowed but copying from other <code>std::simd</code> of the same type will be permitted.                                                                                                  |
| <code>basic_simd(Gen&amp;&amp;)</code>                                                                      | Copy           | Each invocation of the generator builds an individual scalar value of the element type which is bitwise copied into the respective element.                                                                                                                                                                                                                                                                     |
| <code>basic_simd(Iter)</code>                                                                               | Copy/Algorithm | When <code>Iter::value_type</code> is the same, copy the bits.<br>When <code>Iter::value_type</code> is different and a customization point exists for the conversion, create the <code>std::simd</code> as the value iterator type first, and then copy the bits. No customization point will be allowed since it is unlikely that it brings any performance benefit, although this decision can be revisited. |
| Copy functions                                                                                              |                |                                                                                                                                                                                                                                                                                                                                                                                                                 |

| Function                                                                      | Type           | Notes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------------------------------------------------------------------------|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>copy_to</code>                                                          | Copy/Algorithm | <p>When the destination type is the same use a direct bit copy from the <code>simd</code> into memory.</p> <p>When the destination type is different, convert to a <code>std::simd</code> of the destination type and invoke <code>copy_to</code> on that. A customization point is not provided since it is highly unlikely that any hardware support is available for copying to an special type.</p> <p>If the destination type is different and there is no conversion customization point remove the conversion-copy from the overload set.</p> |
| <code>copy_from</code>                                                        | Algorithm      | Equivalent to calling <code>basic_simd(Iter)</code> and performing an assignment.                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Subscript operators                                                           |                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <code>operator[]</code>                                                       | Copy           | Bitwise copy from element into the scalar output value                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Unary operators                                                               |                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <code>operator-</code>                                                        | Custom         | <p>If a customization point or builtin-type support is available use that.</p> <p>Otherwise if <code>operator-</code> is available use <code>simd&lt;T&gt;() - *this</code>.</p> <p>Otherwise remove from the overload set.</p>                                                                                                                                                                                                                                                                                                                      |
| <code>operator--/++</code>                                                    | Algorithm      | <p>If <code>operator+/-</code> are available use <code>*this +/- T(1)</code>.</p> <p>Otherwise remove from the overload set.</p>                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <code>operator!</code>                                                        | Custom         | <p>If a customization point or builtin-type support is available use that.</p> <p>Otherwise if <code>operator==</code> is available return <code>*this == simd&lt;T&gt;()</code>.</p> <p>Otherwise remove from the overload set.</p>                                                                                                                                                                                                                                                                                                                 |
| <code>operator~</code>                                                        | Basis          | <p>If a customization point or builtin-type support is available use that.</p> <p>Otherwise remove from the overload set.</p>                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Binary operators                                                              |                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <code>operator+,-,*,/,%,&lt;&lt;, &gt;&gt;, &amp;,  , ^</code>                | Algorithm      | <p>If a customization point or builtin-type support is available use that.</p> <p>Otherwise remove the <code>simd::operatorX</code> from the overload set.</p>                                                                                                                                                                                                                                                                                                                                                                                       |
| Compound assignment operators                                                 |                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <code>operator+=, -=, *=, /=, %=, &amp;=,  =, ^=, &lt;&lt;=, &gt;&gt;=</code> | Algorithm      | <p>If a customization point or builtin-type support is available for the underlying operation use <code>*this = *this OP rhs</code>.</p> <p>Otherwise remove this from the overload set.</p>                                                                                                                                                                                                                                                                                                                                                         |
| Relational operators                                                          |                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <code>operator==</code>                                                       | Basis          | <p>If a customization point or builtin-type support is available call that.</p> <p>Otherwise remove this from the overload set.</p> <p>Note that although each element represents its values using a specific copyable bit pattern this doesn't mean that the same bit pattern represents an equal value (e.g., floating point NaN bit patterns will never be equal).</p>                                                                                                                                                                            |
| <code>operator!=</code>                                                       | Custom         | <p>If a customization point or builtin-type support is available call that.</p> <p>Otherwise if <code>operator==</code> is provided, return the negation of that function.</p> <p>Otherwise remove from the overload set.</p>                                                                                                                                                                                                                                                                                                                        |
| <code>operator&lt;,&lt;=,&gt;,&gt;=</code>                                    | Basis          | <p>If a customization point or builtin-type support is available call that.</p> <p>Otherwise remove from the overload set.</p>                                                                                                                                                                                                                                                                                                                                                                                                                       |

| Function                                                         | Type      | Notes                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------------------------------------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                  |           | Note that a minimal set could be provided (e.g., <code>operator&lt;</code> and <code>operator==</code> since everything else can be built from those).                                                                                                                                                                                                                                                                                                               |
| Conditional operator                                             |           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <code>simd_select</code>                                         | Copy      | Conditionally copy element bits with no interpretation.                                                                                                                                                                                                                                                                                                                                                                                                              |
| Permute                                                          |           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <code>permute</code> /permute-like                               | Copy      | All permutes (generated or dynamic) move bits from one location to another without interpretation. Related operations like <code>resize</code> , <code>insert</code> , <code>extract</code> work in the same way.                                                                                                                                                                                                                                                    |
| <code>compress/expand</code>                                     | Copy      | All compression and expansion operations move bits from one location to another without interpretation.                                                                                                                                                                                                                                                                                                                                                              |
| <code>gather_from</code>                                         | Copy      | Gather the values as though they were a <code>simd&lt;custom-storage-type&gt;</code> and then use <code>std::bit_cast</code> to convert (at no cost) into a <code>std::simd</code> of the user defined type.                                                                                                                                                                                                                                                         |
| <code>scatter_to</code>                                          | Algorithm | If the same type, bitwise scatter individual elements to the range using direct bitwise copy.<br>If the destination type is different construct a <code>std::simd</code> of the destination type and perform a scatter on that type instead.                                                                                                                                                                                                                         |
| Reductions                                                       |           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <code>reduce</code>                                              | Algorithm | All reduction operations can be implemented using a sequence of permutes and arithmetic operations. If the desired operation for the reduction step is not available in the overload set then the corresponding reduction is also removed from the overload set.<br>No customization point will be provided for reductions since it is unlikely that custom types will have hardware support for reductions. These can be added later if this is found to be untrue. |
| Free functions                                                   |           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <code>min</code><br><code>max</code><br><code>clamp</code>       | Custom    | If the user provides their own ADL overloaded customization point for this function then that will be used.<br>Otherwise if relational operators are available for the type, use those to synthesise this operation (i.e., <code>simd_select(a &lt; b, a, b)</code> for <code>min</code> ).<br>Otherwise remove from the overload set.                                                                                                                               |
| <code>abs</code><br><code>sin</code><br><code>log</code><br>etc. | Custom    | For any other free functions an ADL overload can be provided by the user to handle that specific type.                                                                                                                                                                                                                                                                                                                                                               |

## § 2.1. Required customization points

In the table of function classifications above we can discount the Copy functions from any further thought in this proposal. By limiting `std::simd` elements to those which are trivially copyable, we can provide any sort of operation which moves bits around using the `std::simd` implementation itself, with no special consideration for user-defined types.

Unsurprisingly, the table above shows us that we need customization points for all numeric operations, including:

|                     |                      |               |                  |
|---------------------|----------------------|---------------|------------------|
| plus<br>divides     | minus<br>modulus     | negate        | multiplies       |
| bit AND             | bit OR               | bit NOT       | bit XORr         |
| equal to<br>greater | not equal to<br>less | less or equal | greater or equal |
| logical AND         | logical OR           | logical NOT   |                  |
| shift_left          | shift_right          |               |                  |

Note that also unsurprisingly, these names are all those of the C++ transparent template wrappers, with the exception of shift-left and shift-right. As a small aside, it isn't clear why transparent operators are not provided for shift operations, and perhaps they should be added in for completeness in the future.

We don't require every one of those operators to be defined in order for a `simd<custom-type>` to be instantiated. When a customisation point is not defined for one of the named operations given above then its equivalent operators (or compound assignment operator) will be removed from the overload set. For example, a user-defined complex type might provide addition, subtraction and multiplication, but remove modulus, relational operators, and bitwise operators from the overload set, along with any other operations which depend upon those (e.g., compound modulus, compound bitwise).

The only other customization points needed are conversion functions.

#### Convert to user-defined type from standard type

```
simd<UserDefined>(&simd<standard-type>)
```

#### Convert from user-defined type to standard type

```
simd<standard-type>(&simd<UserDefined>)
```

Note that even if there is a conversion available for the scalar user-defined elements already, we should not use that to build a `std::simd` conversion by iterating through each element since that might be inefficient. If no `simd`-enabled conversions are provided (or perhaps they are only provided in one direction) then the `simd<UserDefined>` is still usable, but it won't allow conversions as part of selected functions such as `copy_to`, `copy_from`, `gather_from` or `scatter_to`, and will require users to load and store `simd` objects only to memory regions of the correct type.

## § 3. Creating a customization framework

There are three main parts of a framework for user-defined element types: storage, unary/binary operators and conversions, and free-functions. In this section we shall look at each of these in turn.

We have a choice of whether to require a user-defined type to opt-in to `std::simd` support or not. Without explicit opt-in from a type, then it becomes possible to create a `simd<SomeType>` from any type with minimal effort, and the default implementation of code within `std::simd` will provide a bare minimum of support for that type. However, the limit of what is automatically provided will not be clear to the user, and whatever generic operations are provided may not be correct for an arbitrary type. To prevent misuse we propose to require an opt-in mechanism for user-defined element support. This allows us to be confident that when a `std::simd<MyType>` is instantiated that consideration has been given to the various permitted operations on that type.

There are many different ways to implement customization points, including template specialization, CPO, or `tag_invoke`. Which mechanism is most suitable can be discussed further if necessary but for this paper proposal we only care about whether customization should be allowed, not the exact mechanism that will be used.

### § 3.1. Storage

In order to perform Copy-like operations on `std::simd` elements we need to be able to inform `std::simd` values of how to store and move the underlying elements. To that end we provide a trait specialization which allows `std::simd` to map from a user-defined type, to the storage that will be used. For example:

```
// Provided by simd
template<typename T> struct simd_custom_type_storage;

// Provided by user for their type
template<> struct simd_custom_type_storage<user-defined-type> {
    using value_type = /* some bit container */;
};
```

The presence of this storage type also provides an opt-in which gates whether a `std::simd` of the user-defined type is permitted.

An alternative (to replace or complement the above) would be to have the user-defined type have a named trait:

```
struct user-defined-type {
    using simd_storage_type = /* some container */;
};
```

In either case, the storage type must have the same size as the user-defined type and must be trivially copyable from one to the other to allow the bits to be moved using `std::bit_cast`. This also implies that bits can be moved to and from a `simd<storage-type>` and a `simd<user-defined-type>`. It is this last feature that allows many of the Copy or Algorithm functions to work. For their purpose they behave like a `std::simd<storage-type>` and then convert to or from a `std::simd<UserType>` using a `std::bit_cast` where required.

A concept which queries whether a user-defined type can be stored in a `std::simd` using the presence or absence of this custom storage trait will also be provided.

### § 3.2. Unary and binary operator customization points

All of the operators for `std::simd` are friend functions in order to allow ADL. We must leave these friend function operators, but allow them to defer to a customization point as required. One possible pseudo-implementation of an individual operator may be this:

```
constexpr friend basic_simd operator+(const basic_simd& lhs, const basic_simd& rhs)
requires (details::simd_has_custom_binary_plus || details::element_has_plus)
{
    if constexpr (details::simd_has_custom_binary_plus)
        return simd_binary_plus(lhs, rhs);
    else
        /* impl-defined */
}
```

In this example `operator+` is only put in the overload set if a builtin-arithmetic type supports addition, or in the case of a user-defined type that the programmer has provided a customisation point. Internally, the function will then invoke either the builtin-simd addition operator or the customisation point function.

We need to specify what the customisation points will be called to allow them to be discoverable. In the example above we have explicitly named the customisation function `simd_binary_plus`. This has the advantage that it is very clear and unambiguous in what it does, but it does introduce potential for high levels of duplication. This is because every operator will have its own unique customisation point name.

An alternative is to exploit the transparent templates that already exist in C++ and use them to differentiate between operations. Here is an example of the signature of a customisation point for a user defined type:

```
template<typename Abi, typename CustomType, std::invocable<CustomType, CustomType> Fn>
constexpr auto
simd_binary_op(const basic_simd<CustomType, Abi>& lhs,
               const basic_simd<CustomType, Abi>& rhs,
               Fn op);
```

This has a unique and distinctive name to mark it as a customisation point, and as a binary operator it takes in two `simd<CustomType>` inputs. It also takes a third parameter which specifies what binary operation to perform, chosen from the list of standard template wrappers. For example, the call site in `operator+` would look like this:

```
return simd_binary_op(lhs, rhs, std::plus<>());
```

Similarly, unary operators can be customized using a customization function called `simd_unary_op` which accept a unary transparent template wrapper.

The advantage of using this mechanism rather than named functions for every required operator is that it removes the need for many different functions, and allows related operations to be consolidated into a single function. It also allows the transparent operator itself to be invoked directly to perform an operation. For example, suppose we want to define a customisation point for a user defined type that has non-standard behaviour for multiply and divide, but everything else works like a standard arithmetic operator (examples of such types include complex numbers and fixed-point numbers). The following pseudo-implementation captures this behaviour:

```
template<typename Abi, typename CustomType, std::invocable<CustomType, CustomType> Fn>
constexpr auto
simd_binary_op(const basic_simd<CustomType, Abi>& lhs,
               const basic_simd<CustomType, Abi>& rhs,
               Fn op)
{
    // Special case for some operators
    if constexpr (std::is_same_v<Fn, std::multiplies<>>) return doCustomMultiply(lhs, rhs);
    else if constexpr (std::is_same_v<Fn, std::divides<>>) return doCustomDivides(lhs, rhs);
    // All other cases defer to an integer instead.
    else return op(simd<int>(lhs), simd<int>(rhs));
}
```

This is not only less verbose but it also makes it obvious how and why the custom type has to be handled differently to a builtin-type.

Unfortunately shift operators don't have transparent wrappers, so if we did use this approach we need one of the following too:

- a specially named customization point (e.g., `simd_shift_left_op`)
- an additional transparent operator added to `std::simd` to allow the existing binary operation to be used (e.g., `std::simd_shift_left<>`)

- a standardised `shift_left` transparent operator (i.e., `std::shift_left<>`)

In the Intel example implementation we have used the second of these. Having a different name and mechanism for shifts introduces extra complexity and non-uniformity. We hope that a transparent operator wrapper for shift might be added in future, in which case it will also be easier to transition to using that if we provide a local alternative to begin with.

### § 3.3. Overloads for free-functions

Anything outside `std::simd` itself can be freely overloaded for the custom type. For example, `abs` could be provided as follows:

```
template<typename Abi>
constexpr auto abs(const basic_simd<fixed_point_16s8, Abi>& v) {
    return /* special-abs-impl */;
}
```

No `std::simd`-specific customization points are required for any of the other functions as overloads will suffice.

## § 4. Implementation Experience

In Intel's implementation of `std::simd`, the customization points described in this proposal have been implemented so that we can use instantiations of `std::simd` which use signal processing data types such as fixed-point or saturating integrals.

Consider what might be needed to make a saturating data type. To begin with, we might already have a 16-bit scalar saturating data type:

```
struct saturating_int16
{
    saturating_int16(int v) : data(v) {}
    int16_t data;

    // Etc.
};
```

We wish to make it possible to create and use `std::simd` values of this type, namely `std::simd<saturating_int16>`. Firstly we need to define the storage, and to opt-in to allowing `std::simd` to vectorise this custom type. We define the following:

```
template<> struct simd_custom_element_type<saturating_int16> { using value_type = int16_t;
```

With this in place, a `std::simd<saturating_int16>` will have exactly the same bit layout as `std::simd<int16_t>`.

Now we can immediately start using many of the existing `std::simd` features.

| C++ Code                                                                                                                                    | Assembly                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <pre>auto broadcast(int16_t x) {     return simd&lt;saturating_int16&gt;(x); }</pre>                                                        | <pre>broadcast(short):     vpbroadcastw    zmm0, edi     ret</pre>                            |
| <pre>auto iq_swap(const simd&lt;saturating_int16&gt;&amp; v) {     return permute(v, [](auto idx) {         return idx ^ 1;     }); }</pre> | <pre>iq_swap([...]&gt; const&amp;): #     vprold    zmm0, zmmword ptr [rdi], 16     ret</pre> |

Next let us add some basic arithmetic operations. Here is a very simple implementation which runs on an Intel AVX2 machine. It could be extended to work on all Intel instruction sets (e.g, AVX-512) but that is outside the scope of this proposal.

```
template<typename Abi>
constexpr auto simd_binary_op(const xvec::basic_simd<saturating_int16, Abi>& lhs,
                              const xvec::basic_simd<saturating_int16, Abi>& rhs,
                              std::plus<>)
{
    auto r = _mm256_adds_epil6(static_cast<__m256i>(lhs), static_cast<__m256i>(rhs));
    return basic_simd<saturating_int16, Abi>(r);
}
```

For brevity we only show the addition operator, but the others can all be provided in similar ways.

Once we have the customization defined for addition it becomes possible to perform addition on `simd<saturating_int16>`, and also perform related operations such as compound-assignment addition, incrementing and even complicated extensions like `reduce`.

| C++ Code                                                                                                                                                | Assembly                                                                                                                                                                                                                                                              |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> auto add(simd&lt;saturating_int16&gt; lhs,         simd&lt;saturating_int16&gt; rhs) {     return lhs + rhs; } </pre>                             | <pre> add([...]): #     vpaddsw ymm0, ymm0, ymm1     ret </pre>                                                                                                                                                                                                       |
| <pre> auto compound_add(simd&lt;saturating_int16&gt; lhs,                  simd&lt;saturating_int16&gt; rhs) {     lhs += rhs;     return lhs; } </pre> | <pre> compound_add([...]): #     vpaddsw ymm0, ymm0, ymm1     ret </pre>                                                                                                                                                                                              |
| <pre> auto increment(simd&lt;saturating_int16&gt; v) {     return ++v; } </pre>                                                                         | <pre> increment([...]): #     vpaddsw ymm0, ymm0, ymmword ptr [.LCPI4     ret </pre>                                                                                                                                                                                  |
| <pre> auto reduce_add(simd&lt;saturating_int16&gt; v) {     return reduce(v, std::plus&lt;{}&gt;()); } </pre>                                           | <pre> reduce_add([...]):     vextracti128    xmm1, ymm0, 1     vpaddsw xmm0, xmm0, xmm1     vpshufd xmm1, xmm0, 238     vpaddsw xmm0, xmm0, xmm1     vpshufd xmm1, xmm0, 85     vpaddsw xmm0, xmm0, xmm1     vpextrw ecx, xmm0, 1     vmovd  eax, xmm0     ret </pre> |

The other numeric and bitwise operators can be defined in similar ways.

Next, we need to be able to compare saturated values. Since a saturated value which is at-rest is essentially a signed integer of the same bit size, we can forward relational operators to their equivalent integers. This demonstrates an advantage of using the transparent operator wrappers, rather than named customization points for each operation; the code can use the strategy to apply to several related operators:

```

template<typename Abi, RelationalOp Fn>
constexpr auto simd_binary_op(const xvec::basic_simd<saturating_int16, Abi>& lhs,
                             const xvec::basic_simd<saturating_int16, Abi>& rhs,
                             Fn)
{
    auto lhsAsInt16 = simd_bit_cast<int16_t>(lhs);
    auto rhsAsInt16 = simd_bit_cast<int16_t>(rhs);

    auto r = Fn{}(lhsAsInt16, rhsAsInt16);
    return typename xvec::basic_simd<saturating_int16, Abi>::mask_type(r);
}

```

Note that the function works by interpreting the incoming bits as integer values using a `bit_cast` and doing the appropriate operation using those. It then turns the generated mask into a mask of the correct type once more. With this

function we not only unlock relational comparisons but also functions like `min` or `reduce_min`:

| C++ Code                                                                                                                             | Assembly                                                                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>auto cmp_gt(simd&lt;saturating_int16&gt; lhs,             simd&lt;saturating_int16&gt; rhs) {     return lhs &gt; rhs; }</pre>  | <pre>cmp_lt([...]): #     vpcmpgtw    ymm0, ymm0, ymm1     ret</pre>                                                                                                                             |
| <pre>auto lowest(simd&lt;saturating_int16&gt; lhs,             simd&lt;saturating_int16&gt; rhs) {     return min(lhs, rhs); }</pre> | <pre>lowest([...]): #     vpminsw ymm0, ymm0, ymm1     ret</pre>                                                                                                                                 |
| <pre>auto reduce_lowest(simd&lt;saturating_int16&gt; v) {     return reduce_min(v); }</pre>                                          | <pre>reduce_lowest([...]): #     vextracti128    xmm1, ymm0, 1     vpminsw xmm0, xmm0, xmm1     vpshufd xmm1, xmm0, 238     vpminsw xmm0, xmm0, xmm1     vpshufd xmm1, xmm0, 85     etc...</pre> |

§ 4.1. Summary of implementation experience

In this section we have given a simple example in which we add customization points to a saturating type, and shown how Intel’s example implementation allows us to use the entire rich API provided by `std::simd` with minimal effort.

§ 5. Conclusion

In this proposal we have outlined the basic mechanisms needed to allow user-defined types to be stored and manipulated by `std::simd` values, and crucially, to be able to do so without knowledge of the internal implementation of the `std::simd` library.

§ References

§ Informative References

[P1928R8]  
Matthias Kretz. *std::simd - Merge data-parallel types from the Parallelism TS 2*. 9 November 2023. URL: <https://wg21.link/p1928r8>

[P2663R4]  
Daniel Towner, Ruslan Arutyunyan. *Proposal to support interleaved complex values in std::simd*. 13 October 2023. URL: <https://wg21.link/p2663r4>